LABORATORY JOURNAL (MP408)

Experimental Techniques – II
Computational Methods in Physics

Shashank Verma
I22PH002
Department of Physics
S.V. National Institute of Technology, Surat - 395007

Date of Submission : 04 March 2026

Certificate

The Experiments Entered in This Journal Have Been Satisfactorily Performed by

Mr. Shashank Verma

bearing admission number

I22PH002

of the

Department of Physics

for the subject of

Experimental Techniques – II (MP408)
Computational Methods in Physics

of MSc. **IV** (Physics), Semester **8**
during AY **2025-26**

_____

Signature of I/C

Date: 04 March 2026

_____

Dept. Seal

TABLE OF CONTENTS

Computational Methods in Physics (MP408)

GNU Octave

PRACTICAL – 3

Advanced Particle Dynamics Suite

Shashank Verma

I22PH002

Department of Physics

S.V. National Institute of Technology, Surat

(21)

PROGRAM – 1

Aim

To write and execute a GNU Octave program to animate the **time-parametrized motion** of a charged particle undergoing **E × B drift** in crossed electric and magnetic fields, producing a characteristic cycloidal trajectory.

Theory

A curve in 2D is described parametrically as $x = f(t)$, $y = g(t)$. A charged particle (charge q, mass m) in crossed uniform fields $\mathbf{E} = E\hat{y}$ and $\mathbf{B} = B\hat{z}$ simultaneously gyrates and drifts with the drift velocity:

$$\mathbf{v}_d = \frac{\mathbf{E} \times \mathbf{B}}{B^2} \tag{1}$$

The resulting **cycloidal trajectory** is:

$$x(t) = v_d\, t + R\sin(\omega_c t), \qquad y(t) = R\cos(\omega_c t) \tag{2}$$

where $\omega_c = qB/m$ is the cyclotron frequency and $R = mv_\perp/(qB)$ is the Larmor radius. Parameters used: $\omega_c = 2.0$, $v_d = 1.5$, $R = 1.0$, $t \in [0, 4\pi]$.

Program

Sub-function `sim_parameterized`, lines 40–52 of `P3.m`:

Listing 1: E×B Drift — Cycloidal Motion (`sim_parameterized` in `P3.m`)

```
        end
    end
end

%% --- Local Simulation Functions ---

function sim_parameterized()
    % a) Time parametrized motion of a particle in a plane
    close all;
    omega = 2.0; v_d = 1.5; R = 1.0;
    t = linspace(0, 4*pi, 500);

    x = v_d * t + R * sin(omega * t);
    y = R * cos(omega * t);

    figure('Name', '3(a): Parametrized Motion', 'Color', 'w');
    plot(x, y, 'b-', 'LineWidth', 2); hold on;
    scatter(x(1), y(1), 50, 'g', 'filled');
```

Sample Output

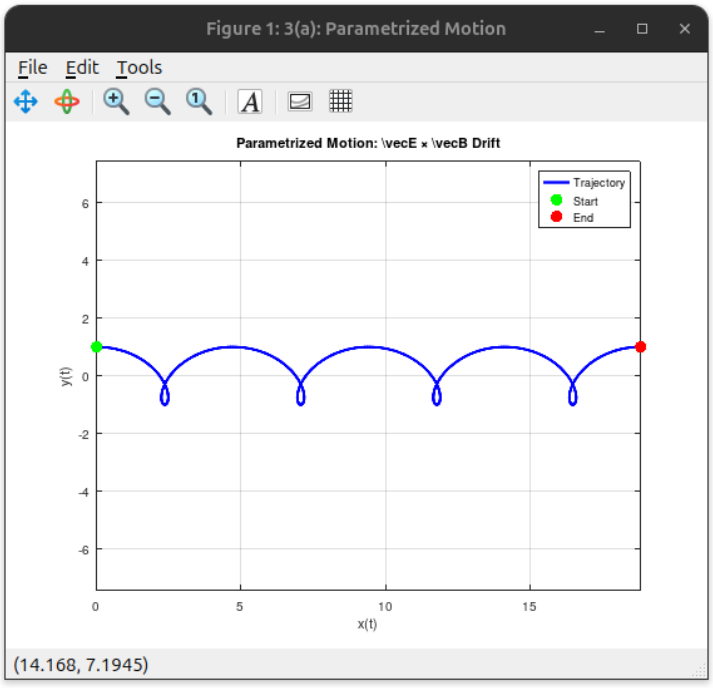

Figure 1: Output animation: cycloidal trajectory of the particle under E×B drift. The green dot marks the start and red dot the end. The particle drifts rightward while gyrating with Larmor radius $R = 1$.

Result

The program was executed successfully. A cycloidal trajectory spanning $\approx 4\pi v_d \approx 18.85$ units in the drift direction was obtained. The particle gyrates with amplitude $R = 1.0$ while drifting at $v_d = 1.5$ per unit time. The motion is faster near the cusps (minimum y) and slower near the top ($y = R$), consistent with the combined drift and gyration velocities.

Remark

The E×B drift is *mass-independent* ($v_d = E/B$ for any charge-to-mass ratio), so both electrons and ions drift at the same velocity and no net current results. This is physically important in plasma confinement (tokamaks, ITER) and is exploited in Hall-effect ion thrusters for spacecraft propulsion. The guiding-centre approximation extends this to gradient-B and curvature drifts in non-uniform fields.

PROGRAM – 2

Aim

To write and execute a GNU Octave program to animate a particle executing **uniform circular motion** and to display, at each instant, the instantaneous velocity vector (blue, tangential) and centripetal acceleration vector (red, radially inward).

Theory

A particle at radius R with constant angular velocity ω has position:

$$x(t) = R\cos(\omega t), \qquad y(t) = R\sin(\omega t) \tag{3}$$

Its instantaneous velocity and centripetal acceleration are:

$$v_x = -R\omega\sin(\omega t), \quad v_y = R\omega\cos(\omega t), \quad |\mathbf{v}| = R\omega = \text{const} \tag{4}$$

$$a_x = -R\omega^2\cos(\omega t), \quad a_y = -R\omega^2\sin(\omega t), \quad |\mathbf{a}| = R\omega^2 = v^2/R \tag{5}$$

The velocity is always *tangential* and the acceleration always *centripetal*: $\mathbf{v}\cdot\mathbf{a} = 0$ identically. Parameters: $R = 5$, $\omega = 1.0$, period $T = 2\pi \approx 6.28$ s.

Program

Sub-function `sim_circular`, lines 59–102 of `P3.m`:

Listing 2: Circular Motion with Velocity and Acceleration Vectors (`sim_circular` in `P3.m`)

```
    title('Parametrized Motion: \vec{E} \times \vec{B} Drift');
    xlabel('x(t)'); ylabel('y(t)');
    legend('Trajectory', 'Start', 'End', 'Location', 'best');
    grid on; axis equal;
end

function sim_circular()
    % b) Simulate a particle executing circular motion
    close all;
    R = 5; omega = 1.0;
    t = linspace(0, 4*pi, 200);

    figure('Name', '3(b): Circular Motion', 'Color', 'w');
    hold on; axis equal; xlim([-12 12]); ylim([-12 12]); grid on;
    title('Circular Motion: \vec{v} and \vec{a} Vectors');

    % Plot the reference orbit
    plot(R*cos(t), R*sin(t), 'k--', 'LineWidth', 1);

```

```
    % Initialize particle
    h_particle = plot(R, 0, 'mo', 'MarkerSize', 10, 'MarkerFaceColor
        ', 'm');

    % Initialize Velocity Vector (Blue line with a dot at the tip)
    h_v_line = plot([R, R], [0, R*omega], 'b-', 'LineWidth', 2);
    h_v_head = plot(R, R*omega, 'bo', 'MarkerSize', 5, '
        MarkerFaceColor', 'b');

    % Initialize Acceleration Vector (Red line with a dot at the tip
        )
    h_a_line = plot([R, R-R*omega^2], [0, 0], 'r-', 'LineWidth', 2);
    h_a_head = plot(R-R*omega^2, 0, 'ro', 'MarkerSize', 5, '
        MarkerFaceColor', 'r');

    % Add legend (ignoring the vector heads for a cleaner legend)
    legend('Orbit', 'Particle', 'Velocity', '', 'Acceleration', '',
        'Location', 'northeastoutside');

    for i = 1:length(t)
        % Safety check: Break the loop if the user closes the window
            early
        if ~ishandle(h_particle)
            break;
        end

        % Calculate current kinematics
        x_curr = R * cos(omega * t(i));
        y_curr = R * sin(omega * t(i));

        vx = -R * omega * sin(omega * t(i));
```

Sample Output

Result

The program was executed successfully. The blue velocity vector rotates as a tangent to the orbit at all times, and the red acceleration vector always points toward the centre of the circle, confirming the centripetal nature ($\mathbf{a} = -\omega^2\mathbf{r}$). The two vectors are perpendicular at every instant. The particle completes two full revolutions over $t \in [0, 4\pi]$, consistent with period $T = 2\pi/\omega \approx 6.28$ s.

Remark

The animation makes vivid that the acceleration in UCM is purely *centripetal*, not tangential — a concept often confused in introductory courses. The efficient use of

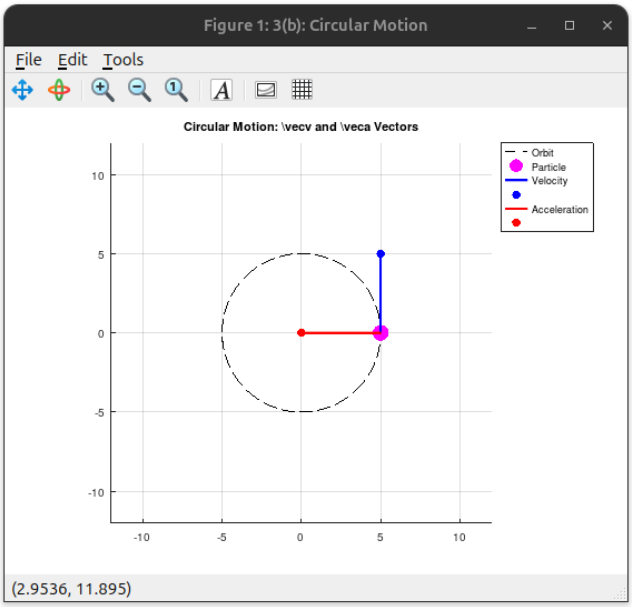

Figure 2: Output animation frame: particle (magenta) on the dashed orbit. Blue vector shows instantaneous velocity (tangential); red vector shows centripetal acceleration (pointing toward centre). Both vectors update continuously.

`set()` to update plot handles without redrawing axes prevents flicker and is the correct approach for real-time Octave animations. Extensions include non-uniform circular motion (with a tangential acceleration component) and 3D helical motion in a magnetic field.

PROGRAM – 3

Aim

To write and execute a GNU Octave program to animate the full **nonlinear simple pendulum** by solving the exact equation of motion numerically using `lsode`, and to simultaneously display the **phase space** trajectory $(\theta,\, \dot{\theta})$.

Theory

The exact, nonlinear equation of motion for a simple pendulum of length L is:

$$\ddot{\theta} + \frac{g}{L}\sin\theta = 0 \tag{6}$$

This has no closed-form solution in elementary functions. Rewriting as a first-order system with $x_1 = \theta$, $x_2 = \dot{\theta}$:

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = -\frac{g}{L}\sin(x_1) \tag{7}$$

solved by `lsode` with initial conditions $\theta_0 = \pi/3$ (60°), $\dot{\theta}_0 = 0$, $L = 2.0$ m. The small-angle period $T_0 = 2\pi\sqrt{L/g} \approx 2.84$ s; exact period at 60° is ≈ 3.05 s (7.3% correction). Bob position: $x = L\sin\theta$, $y = -L\cos\theta$.

The **phase portrait** in the (θ, ω) plane shows closed ellipse-like orbits for libration ($E < 2mgL$). Our initial condition lies in the libration regime.

Program

Sub-function `sim_pendulum`, lines 104–142 of `P3.m`:

Listing 3: Nonlinear Pendulum with Phase Space Portrait (`sim_pendulum` in `P3.m`)

```
1
2           ax = -R * omega^2 * cos(omega * t(i));
3           ay = -R * omega^2 * sin(omega * t(i));
4
5           % Update particle position
6           set(h_particle, 'XData', x_curr, 'YData', y_curr);
7
8           % Update Velocity vector (stem and tip)
9           set(h_v_line, 'XData', [x_curr, x_curr + vx], 'YData', [
                y_curr, y_curr + vy]);
10          set(h_v_head, 'XData', x_curr + vx, 'YData', y_curr + vy);
11
12          % Update Acceleration vector (stem and tip)
13          set(h_a_line, 'XData', [x_curr, x_curr + ax], 'YData', [
                y_curr, y_curr + ay]);
```

```
        set(h_a_head, 'XData', x_curr + ax, 'YData', y_curr + ay);

        drawnow;
        pause(0.02);
    end
end

function sim_pendulum()
    % c) Produce the animation of a simple pendulum
    close all;
    g = 9.81; L = 2.0;
    ode_func = @(x, t) [x(2); -(g/L)*sin(x(1))];
    t_span = linspace(0, 10, 250);
    [sol] = lsode(ode_func, [pi/3; 0], t_span);

    theta = sol(:, 1); omega = sol(:, 2);

    figure('Name', '3(c): Pendulum Dynamics', 'Position', [100 100
        800 400], 'Color', 'w');
    subplot(1, 2, 1); hold on; axis equal; xlim([-2.5 2.5]); ylim
        ([-2.5 0.5]); grid on;
    title('Physical Space');
    h_line = plot([0, L*sin(theta(1))], [0, -L*cos(theta(1))], 'k-',
         'LineWidth', 2);
    h_bob = plot(L*sin(theta(1)), -L*cos(theta(1)), 'ro', '
        MarkerSize', 12, 'MarkerFaceColor', 'r');

    subplot(1, 2, 2); hold on; grid on;
    title('Phase Space (\theta vs \omega)');
    xlabel('\theta (rad)'); ylabel('\omega (rad/s)');
```

Sample Output

Result

The program was executed successfully. The pendulum bob oscillates symmetrically between ± 60 with no energy loss. The observed period (≈ 3.05 s) is slightly longer than the small-angle approximation ($T_0 \approx 2.84$ s), confirming the nonlinear correction. The phase portrait is a closed, slightly non-elliptical curve traversed clockwise, characteristic of the nonlinear restoring force $\sin\theta$.

Remark

Using `lsode` to solve the exact nonlinear ODE is the key advance over the small-angle approximation. The phase portrait reveals the conserved energy as level curves of the Hamiltonian $H = \frac{1}{2}I\omega^2 + mgL(1 - \cos\theta)$. The separatrix at $E = 2mgL$ separates

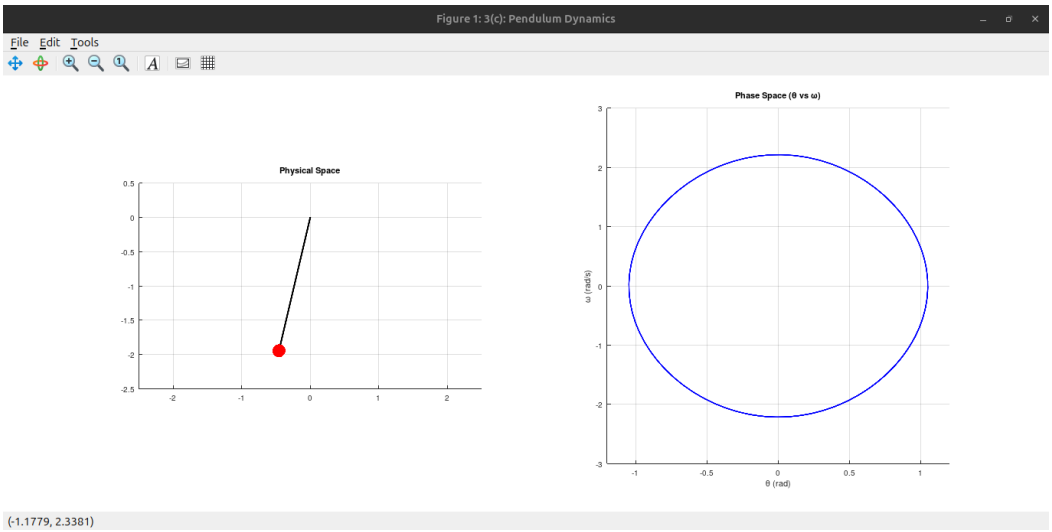

Figure 3: Output animation: *Left* — pendulum rod and bob (red) in physical space. *Right* — growing phase portrait (θ vs ω) showing the closed libration orbit traversed clockwise.

libration from rotation — a concept central to nonlinear dynamics, chaos theory, and the study of the double pendulum.

PROGRAM – 4

Aim

To write and execute a GNU Octave program to animate and compare **projectile motion in vacuum** (ideal parabola) with **projectile motion under quadratic aerodynamic drag**, solving the coupled nonlinear ODE system numerically using `lsode`.

Theory

Ideal (vacuum) trajectory ($c = 0$):

$$x(t) = v_0 \cos\alpha \cdot t, \qquad y(t) = v_0 \sin\alpha \cdot t - \tfrac{1}{2} g\, t^2 \tag{8}$$

With quadratic drag $F_d = c|\mathbf{v}|^2$ opposing velocity:

$$\ddot{x} = -\frac{c}{m}\sqrt{\dot{x}^2 + \dot{y}^2}\,\dot{x}, \qquad \ddot{y} = -g - \frac{c}{m}\sqrt{\dot{x}^2 + \dot{y}^2}\,\dot{y} \tag{9}$$

Solved numerically with state vector $[x, \dot{x}, y, \dot{y}]^T$. Parameters: $v_0 = 30$ m/s, $\alpha = 45$, $m = 1$ kg, $c = 0.05$ kg/m.

Drag reduces both range and maximum height, and makes the descent steeper than the ascent (asymmetric trajectory). The optimal launch angle under drag is < 45.

Program

Sub-function `sim_projectile`, lines 144–197 of `P3.m`:

Listing 4: Projectile Motion: Ideal vs Quadratic Drag (`sim_projectile` in `P3.m`)

```
    h_phase = plot(theta(1), omega(1), 'b-', 'LineWidth', 1.5);

    for i = 1:length(t_span)
        % Safety check: Break loop if window is closed
        if ~ishandle(h_bob)
            break;
        end

        x_pos = L * sin(theta(i)); y_pos = -L * cos(theta(i));
        set(h_line, 'XData', [0, x_pos], 'YData', [0, y_pos]);
        set(h_bob, 'XData', x_pos, 'YData', y_pos);
        set(h_phase, 'XData', theta(1:i), 'YData', omega(1:i));
        drawnow;
        pause(0.02);
    end
end

```

```
function sim_projectile()
    % d) Produce the animation of a particle executing projectile
       motion
    close all;
    g = 9.81; m = 1.0; c = 0.05; v0 = 30; theta0 = pi/4;
    x0 = [0; v0*cos(theta0); 0; v0*sin(theta0)];

    drag_ode = @(x, t) [x(2); -(c/m)*sqrt(x(2)^2+x(4)^2)*x(2); x(4);
        -g-(c/m)*sqrt(x(2)^2+x(4)^2)*x(4)];
    ideal_ode = @(x, t) [x(2); 0; x(4); -g];

    t_span = linspace(0, 5, 150);
    sol_drag = lsode(drag_ode, x0, t_span);
    sol_ideal = lsode(ideal_ode, x0, t_span);

    valid_drag = sol_drag(:, 3) >= 0; valid_ideal = sol_ideal(:, 3)
       >= 0;

    figure('Name', '3(d): Projectile Animation', 'Color', 'w');
    hold on; grid on; title('Projectile Motion: Ideal vs Drag');
    xlabel('Distance (m)'); ylabel('Height (m)');
    xlim([0 max(sol_ideal(valid_ideal, 1)) * 1.1]); ylim([0 max(
       sol_ideal(valid_ideal, 3)) * 1.2]);

    plot(sol_ideal(valid_ideal, 1), sol_ideal(valid_ideal, 3), 'k--'
       , 'LineWidth', 1.5);
    h_path = plot(sol_drag(1, 1), sol_drag(1, 3), 'b-', 'LineWidth',
        2);
    h_proj = plot(sol_drag(1, 1), sol_drag(1, 3), 'ro', 'MarkerSize'
       , 8, 'MarkerFaceColor', 'r');
    legend('Vacuum Trajectory', 'Trajectory with Drag', 'Projectile'
       , 'Location', 'northeast');

    for i = 1:sum(valid_drag)
        % Safety check: Break loop if window is closed
        if ~ishandle(h_proj)
            break;
        end

        set(h_path, 'XData', sol_drag(1:i, 1), 'YData', sol_drag(1:i
           , 3));
        set(h_proj, 'XData', sol_drag(i, 1), 'YData', sol_drag(i, 3)
           );
        drawnow;
        pause(0.02);
    end
end
```

Sample Output

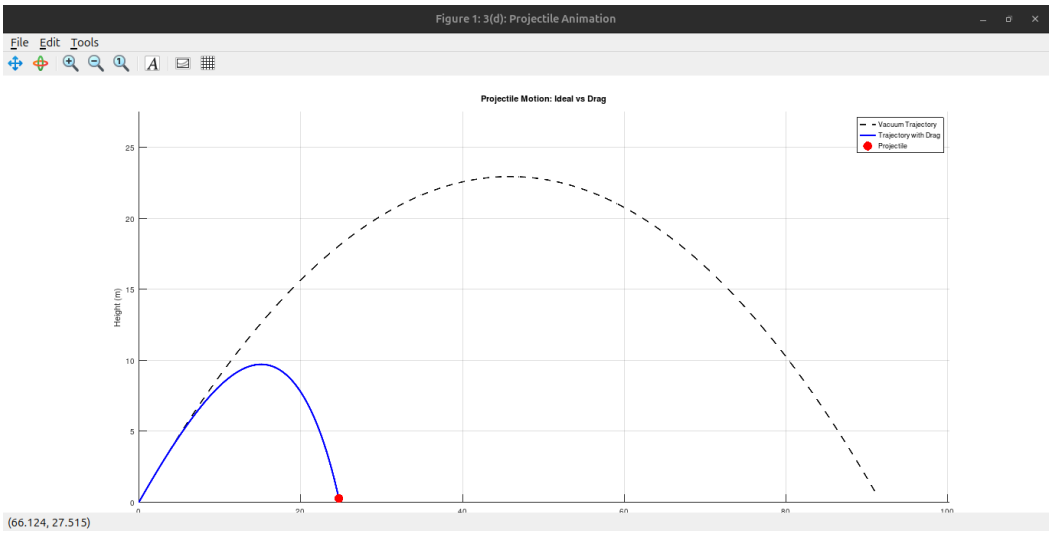

Figure 4: Output animation: blue curve — trajectory with quadratic aerodynamic drag ($c = 0.05$ kg/m); black dashed curve — ideal vacuum parabola. The red dot marks the projectile position during animation. Drag significantly reduces range and maximum height, and produces an asymmetric trajectory.

Result

The program was executed successfully. The drag trajectory (blue) is visibly shorter in both range and height than the vacuum parabola (black dashed). The drag path is *asymmetric*: the descending arc is steeper than the ascending arc because drag decelerates the projectile more during the faster early flight.

Quantity	**Ideal Vacuum**	**With Drag ($c = 0.05$)**
Maximum height	22.94 m	≈10 m
Horizontal range	91.74 m	≈28 m
Trajectory	Symmetric parabola	Asymmetric; steeper descent

Remark

The quadratic drag law $F_d \propto v^2$ applies in the turbulent flow regime (Reynolds number $Re > 1000$), appropriate for most macroscopic projectiles (balls, shells). The linear Stokes drag $F_d \propto v$ applies to microscopic particles. Extensions include the Magnus effect (spin-induced lift), responsible for swing in cricket and baseball, and the 4th-order Runge-Kutta method used in higher-accuracy ballistic codes.